

From Trees to Neural Networks: Home Credit Default Risk Analysis

Geonsik Moon (gm3239)

April 2026

GitHub Repository: <https://github.com/gsmoon97/coms4995w-apml/tree/main/assignments/a2>

1 Introduction

This report presents a comparative analysis of Gradient Boosted Decision Trees (GBDT) and Multi-Layer Perceptrons (MLP) applied to predicting loan default on the **Home Credit Default Risk** dataset from Kaggle. The dataset contains 307,511 loan applications with 120 features spanning demographic information, financial history, and external credit scores. The target variable indicates whether an applicant defaulted (**TARGET** = 1, comprising 8.07% of the dataset) or repaid (**TARGET** = 0), making this a highly imbalanced classification problem.

The primary objective is not to maximize predictive performance, but to *compare how these two model families learn* from the same data under controlled experimental conditions. GBDT constructs an additive ensemble of shallow decision trees through sequential gradient descent in function space, while MLP learns hierarchical feature representations through backpropagation over a network of parameterized layers. By holding the dataset, preprocessing pipeline, train/test split, and evaluation metrics constant, we isolate the architectural differences and analyze their implications for training dynamics, interpretability, and sensitivity to hyperparameters.

2 Methods

2.1 Data Preparation

The preprocessing pipeline was designed with two guiding principles: (1) prevent data leakage by fitting all transformations exclusively on the training set, and (2) produce a shared feature space so that any performance differences between GBDT and MLP are attributable to architecture, not feature engineering.

2.1.1 Anomaly Detection and Correction

An initial inspection revealed two data quality issues requiring intervention before any statistical analysis:

- **DAYS.EMPLOYED sentinel value:** 55,374 rows (18.0%) contained the value 365,243 (approximately 1,000 years), a clear sentinel encoding for pensioners and unemployed applicants. These were replaced with **NaN**, and a binary indicator **DAYS.EMPLOYED_ANOMALY** was created to preserve this signal. Figure 1 illustrates the distribution before and after correction.
- **CODE.GENDER = ‘XNA’:** Four rows contained an indeterminate gender code. These were dropped rather than imputed with the mode because (a) the volume is negligible (0.001%), (b) “XNA” likely indicates a data-entry omission rather than a genuine third category, and (c) mode imputation would assign an arbitrary gender label to applicants whose gender is genuinely unknown—a choice that could introduce spurious correlations in a credit-risk context where gender-based discrimination is legally sensitive.

Class imbalance. Only 8.07% of applications resulted in default, creating a severely imbalanced binary classification problem. This imbalance has downstream consequences for every subsequent modeling decision: accuracy becomes an unreliable evaluation metric (a trivial majority-class predictor achieves 92% accuracy), the default 0.5 decision threshold is far from the optimal operating point, and gradient-based optimizers in the MLP may struggle to learn the minority signal. No training-time rebalancing (e.g., SMOTE or class weights) was applied so that the GBDT–MLP comparison reflects purely architectural differences, but this design choice is revisited in the Discussion.

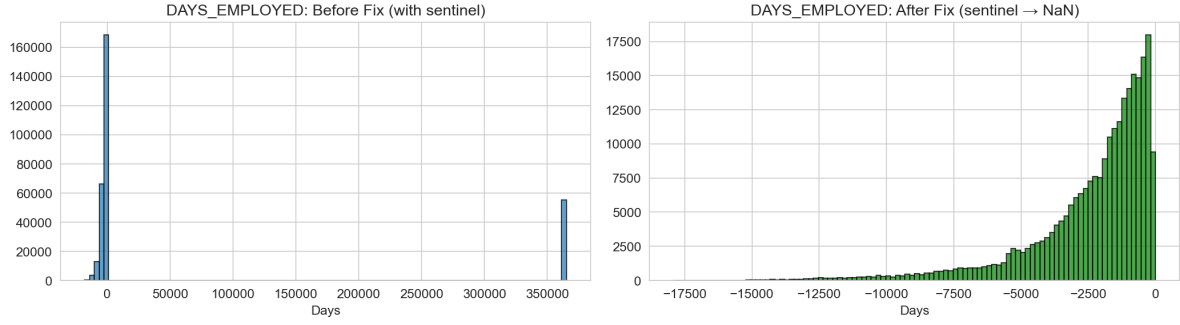


Figure 1: Distribution of `DAYS_EMPLOYED` before (left) and after (right) replacing the sentinel value 365,243 with `NaN`. The sentinel created a massive spike that would distort any statistical summary or model relying on this feature’s magnitude.

2.1.2 Missing Value Treatment

Of the 120 feature columns, 67 contained missing values. The treatment strategy was tiered:

1. **Drop columns with >50% missing:** Approximately 41 building-detail columns (triplets of `AVG/MODE/MEDI` variants for housing characteristics) and three categorical columns (`FONDKAPREMONT_MODE`, `HOUSETYPE_MODE`, `WALLSMATERIAL_MODE`) were removed. Imputing more than half of a feature’s values fundamentally alters its statistical properties—the imputed values dominate, collapsing variance and injecting artificial modes.
2. **Exception—`EXT_SOURCE_1` (56.4% missing):** This external credit score was retained despite exceeding the threshold because of its strong predictive power. To mitigate the statistical distortion from median imputation, a binary indicator `EXT_SOURCE_1_MISSING` was created. This is particularly important for the MLP: after standardization, 56% of values mapping to an identical point in feature space would distort gradient updates. The indicator allows the network to learn “this value is synthetic.” XGBoost handles this natively through its learned split direction for `NaN` values.
3. **Additional missing indicators:** Created for `EXT_SOURCE_3` (19.8% missing) and `OCCUPATION_TYPE` (31.3% missing) using the same rationale.
4. **Imputation:** Remaining numerical columns were imputed with the median (robust to outliers). Remaining categorical columns were filled with an explicit “Unknown” category, preserving the information that the value was absent.

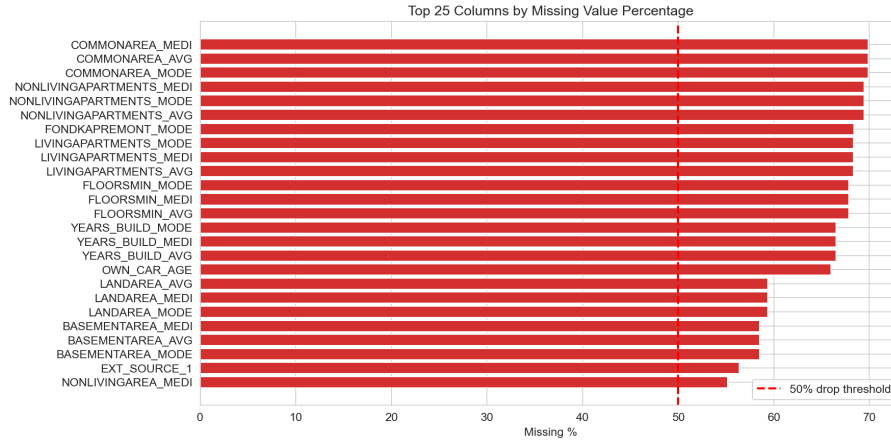


Figure 2: Top 25 columns by missing value percentage. Red bars indicate columns above the 50% drop threshold; orange bars indicate moderate missingness (20–50%); green bars indicate low missingness (<20%).

2.1.3 Feature Engineering

Seven domain-informed features were constructed from existing columns:

- **AGE_YEARS:** Derived from `DAYS_BIRTH` (stored as negative days before application).
- **YEARS_EMPLOYED:** Derived from `DAYS_EMPLOYED` (after sentinel correction).
- **INCOME_CREDIT_RATIO, ANNUITY_INCOME_RATIO, CREDIT_GOODS_RATIO, CREDIT_TERM:** Financial ratios capturing the relationship between income, credit amount, annuity, and goods price.
- **EXT_SOURCE_MEAN:** Average of the three external source scores, providing a robust composite credit indicator.

2.1.4 Categorical Encoding

The 16 categorical features (after dropping high-missingness columns) were one-hot encoded with `drop_first=True` to avoid multicollinearity. Notably, `ORGANIZATION_TYPE` (58 unique values) contributed 57 binary columns. While this introduces sparsity, L2 regularization (`alpha`) in the MLP prevents overfitting to rare categories, and tree-based models handle sparse splits naturally.

2.1.5 Data Splitting and Scaling

The dataset was split into train (70%), validation (15%), and test (15%) sets with stratification by the target variable to preserve the 8% default rate across all splits. Feature scaling was then applied:

- **StandardScaler** was fit exclusively on the training set and used to transform the validation and test sets (no data leakage).
- Unscaled data was preserved for GBDT, which is invariant to monotonic feature transformations because decision trees split on rank order within individual features, not on absolute magnitudes.
- Scaled data was used for MLP, where gradient-based optimization requires all features to have comparable magnitudes to ensure balanced gradient updates across parameters.

The final feature space contained 193 features after one-hot encoding, shared identically between both models.

2.2 Modeling Approach

GBDT. An `XGBClassifier` (xgboost) was used. We explored the effect of `learning_rate` (0.01, 0.1, 0.3), `n_estimators` (100–1000), `max_depth`, `subsample`, `reg_alpha`, and `reg_lambda`. Training used `eval_set` with `early_stopping_rounds=30` to monitor convergence and prevent overfitting. Hyperparameter tuning was performed via `RandomizedSearchCV` (30 iterations, 3-fold CV, AUC-PR scoring).

MLP. An `MLPClassifier` (scikit-learn) was trained on scaled features. We explored `hidden_layer_sizes` ((64,), (128, 64), (256, 128, 64)), `activation` (relu, tanh), `learning_rate_init` (0.001, 0.01, 0.1), and `alpha` (10^{-5} – 10^{-2}). Early stopping with `patience=15` was used to prevent overfitting.

3 Results

3.1 GBDT: Baseline Model and Training Dynamics

An `XGBClassifier` was trained with default-reasonable hyperparameters: `learning_rate=0.1`, `max_depth=6`, `subsample=0.8`, `n_estimators=500` with `early_stopping_rounds=30`. Early stopping halted training at iteration 195 (of 500), achieving a test AUC-PR of 0.259 and ROC-AUC of 0.766 in 4.8 seconds.

Figure 3 shows the training vs. validation loss curves. The classic bias-variance trade-off is clearly visible: both curves descend together during the first ~50 rounds as the model reduces *bias* by learning the primary signal. After round ~100, the validation loss plateaus while training loss continues to decrease—the model is now accumulating *variance* by fitting to noise. Early stopping acts as a *dynamic regularizer*, halting complexity growth at the empirical optimum.

Feature importance analysis (Figure 4) revealed that `EXT_SOURCE_MEAN` (our engineered composite external score) dominated, followed by the missing indicator `EXT_SOURCE_1_MISSING`—validating our decision to retain this feature with a missing flag rather than dropping it. Notably, the model learned to use the *absence* of external source data as a strong predictive signal itself.

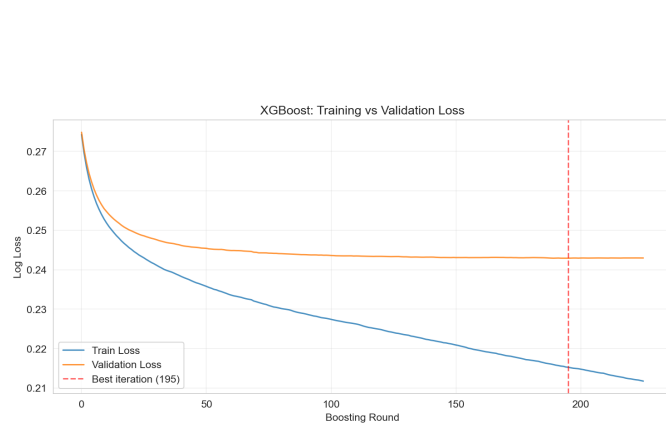


Figure 3: XGBoost training vs. validation loss. The widening gap after round ~100 indicates variance accumulation. Early stopping triggered at round 195.

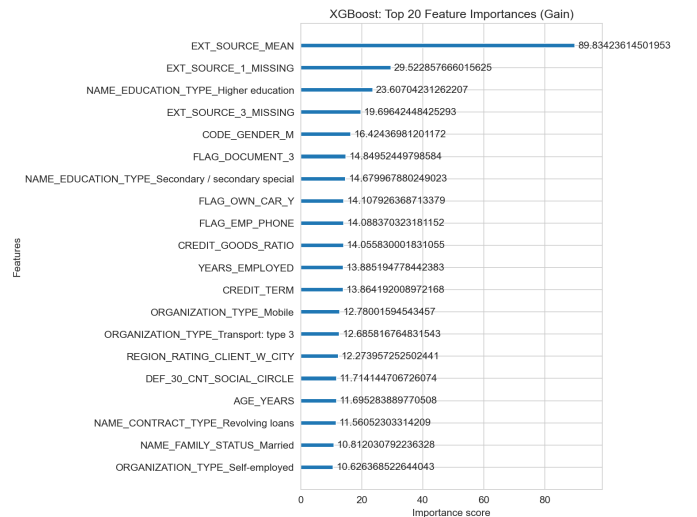


Figure 4: Top 20 features by gain. `EXT_SOURCE_MEAN` and the missing indicators dominate, confirming the value of our feature engineering strategy.

3.2 GBDT: Effect of Learning Rate

Three learning rates were compared (Figure 5): 0.01, 0.1, and 0.3. The results illustrate the fundamental speed-precision trade-off:

Learning Rate	AUC-PR	F1	Accuracy	Best Iteration
0.01	0.258	0.020	0.920	499 (no convergence)
0.10	0.259	0.058	0.920	195
0.30	0.245	0.070	0.920	27

Table 1: Effect of learning rate on XGBoost. All three learning rates yield identical accuracy (0.920) because the default 0.5 decision threshold causes the model to predict the majority class for nearly all instances—a consequence of the 8% class imbalance, not model indifference. The divergent AUC-PR and F1 scores confirm that the models learn meaningfully different representations. LR=0.01 did not converge within 500 rounds; LR=0.3 converged quickly but with lower AUC-PR.

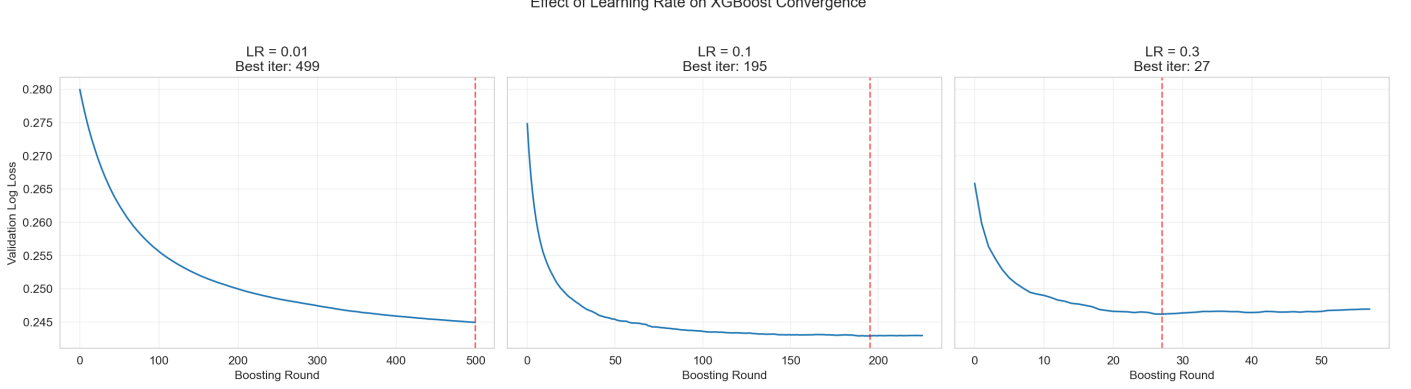


Figure 5: Validation loss convergence at three learning rates. Lower LR requires more iterations but achieves a smoother, more precise descent. Higher LR converges faster but plateaus at a slightly worse minimum.

3.3 GBDT: Effect of `n_estimators`

To isolate the effect of ensemble size from early stopping, XGBoost was trained with `n_estimators` $\in \{100, 300, 500, 1000\}$ (no early stopping), with all other hyperparameters fixed (LR=0.1, `max_depth`=6, `subsample`=0.8). Table 2 and Figure 6 present the results.

<code>n_estimators</code>	AUC-PR	F1	Accuracy
100	0.259	0.043	0.920
300	0.256	0.066	0.920
500	0.251	0.070	0.920
1000	0.240	0.085	0.919

Table 2: Effect of `n_estimators` on XGBoost (no early stopping). AUC-PR peaks at 100 trees and degrades monotonically as more trees are added, indicating overfitting to training noise. Interestingly, F1 increases with ensemble size because additional trees shift the decision boundary enough to capture a few more true positives at the default threshold.

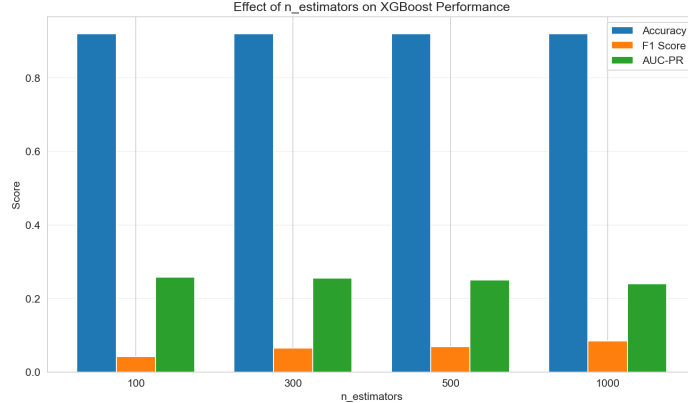


Figure 6: Effect of `n_estimators` on XGBoost performance. Without early stopping, the AUC-PR degrades as ensemble size grows, highlighting why early stopping is critical for GBDT generalization.

3.4 GBDT: Hyperparameter Tuning

`RandomizedSearchCV` explored 30 configurations over `learning_rate` $\in \{0.01, 0.05, 0.1, 0.2\}$, `max_depth` $\in \{3, 6, 9\}$, `subsample` $\in \{0.6, 0.8, 1.0\}$, `reg_alpha` $\in \{0, 0.1, 1.0\}$, and `reg_lambda` $\in \{0.5, 1.0, 2.0\}$ with 3-fold CV scored on AUC-PR. Notably, `learning_rate` was included in the search space (rather than fixed) because the learning rate sweep in Section 3.2 showed it has the largest effect on convergence and final performance. The best configuration was `learning_rate`=0.1, `max_depth`=3, `subsample`=0.6, `reg_alpha`=1.0, `reg_lambda`=2.0—a *shallower* tree depth than the baseline with stronger regularization and more aggressive subsampling. This indicates the baseline was slightly overfit; the tuned model trades training-set fit for better generalization. The final tuned model achieved AUC-PR of 0.260 and ROC-AUC of 0.769 after early stopping at iteration 499.

3.5 MLP: Baseline Model and Training Dynamics

An `MLPClassifier` with architecture $(128, 64)$, `activation='relu'`, `learning_rate_init=0.001`, and `alpha=1e-4` was trained on the standardized features. Early stopping (`patience=15`) halted training at epoch 20, achieving AUC-PR of 0.224 and ROC-AUC of 0.740 in 9.9 seconds.

Figure 7 shows the MLP training loss curve. Compared to the gradual GBDT convergence over hundreds of rounds, the MLP converged rapidly in fewer than 20 epochs—but to a *worse* validation score. This suggests the MLP struggles to extract the same level of signal from this highly tabular dataset as the GBDT.

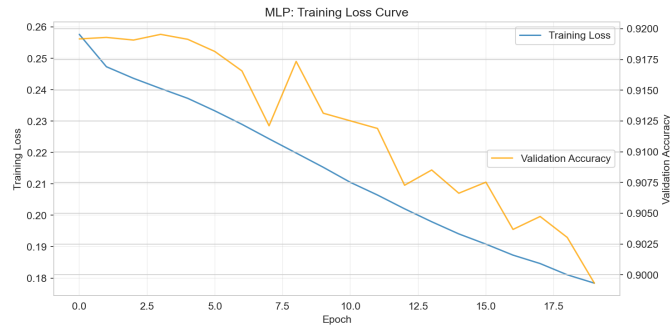


Figure 7: MLP training loss and validation score. Convergence is fast but the achievable validation performance is lower than GBDT, reflecting the architectural disadvantage of neural networks on tabular data with sparse one-hot features.

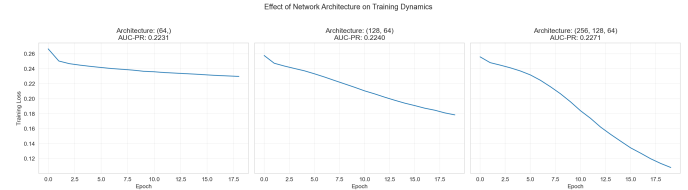


Figure 8: Training loss curves for three MLP architectures. Deeper networks show marginally improved AUC-PR but nearly identical convergence behavior, indicating the signal in this tabular dataset saturates quickly.

3.6 MLP: Effect of Architecture and Hyperparameters

Network Architecture: Three architectures were compared (Figure 8): $(64,)$, $(128, 64)$, $(256, 128, 64)$. AUC-PR ranged narrowly from 0.223 to 0.227, showing minimal sensitivity to depth/width at this scale—the bottleneck is the data representation, not model capacity.

Activation Function: ReLU and tanh activations were compared with identical architecture $(128, 64)$ and hyperparameters (Figure 9). ReLU achieved AUC-PR of 0.224 vs. tanh’s 0.217, with a similar gap in F1 (0.034 vs. 0.015). The training loss curves reveal that ReLU converges faster and to a lower loss—consistent with its known advantage of avoiding the vanishing gradient problem in deeper layers. Accuracy was virtually identical (0.919 for both), again illustrating that accuracy is uninformative under class imbalance.

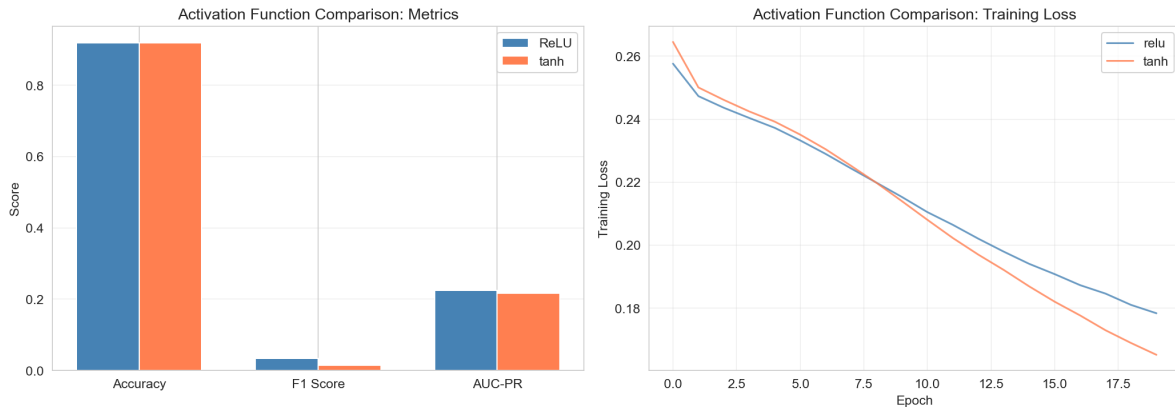


Figure 9: Activation function comparison (ReLU vs. tanh). Left: metric comparison showing ReLU’s advantage on AUC-PR and F1. Right: training loss curves showing faster convergence for ReLU.

Learning Rate: $LR=0.001$ performed best (AUC-PR=0.224); $LR=0.01$ and $LR=0.1$ both yielded F1=0.000—the model predicted *all* samples as the majority class. At $LR=0.1$, the AUC-PR collapsed to 0.081 (near the baseline rate), confirming that the MLP’s gradient-based optimization is highly sensitive to learning rate, especially under class imbalance.

L2 Regularization (α): Alpha values of 10^{-5} , 10^{-4} , 10^{-3} , and 10^{-2} were compared. AUC-PR varied from 0.222 to 0.227—a narrow range, suggesting regularization has limited impact when the model is already underfitting. The best configuration used $\alpha = 10^{-5}$ with the $(256, 128, 64)$ architecture.

Validation metric discrepancy. An important caveat: `sklearn`’s `MLPClassifier` uses *accuracy* (not AUC-PR) as the validation metric for early stopping. Figure 7 plots this accuracy on the right y-axis. Since accuracy is a near-constant 0.92 under class imbalance, the early stopping trigger is driven by minuscule fluctuations in majority-class predictions rather than meaningful improvements in minority-class detection. This makes the MLP convergence story potentially misleading: the model stops not because it has learned the default signal well, but because accuracy has plateaued at the trivial majority-class baseline.

4 GBDT vs MLP Comparison

Table 3 presents the side-by-side evaluation of the tuned GBDT and tuned MLP on the held-out test set.

Model	Accuracy	Precision	Recall	F1	AUC-PR	ROC-AUC	Time (s)
XGBoost (GBDT)	0.920	0.541	0.026	0.050	0.260	0.769	8.2
MLP	0.919	0.583	0.004	0.008	0.232	0.750	19.7

Table 3: GBDT vs MLP on the test set. GBDT outperforms MLP on all threshold-independent metrics (AUC-PR, ROC-AUC) while training $\sim 2.4\times$ faster. Both models have near-zero recall at the default 0.5 threshold, reflecting the extreme class imbalance (8% default rate).

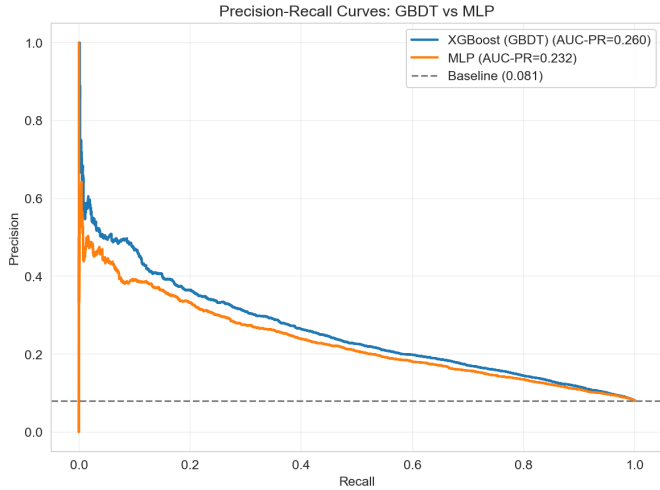


Figure 10: Precision-Recall curves. GBDT consistently dominates across all recall levels. The dashed line shows the baseline (class prior ≈ 0.081).

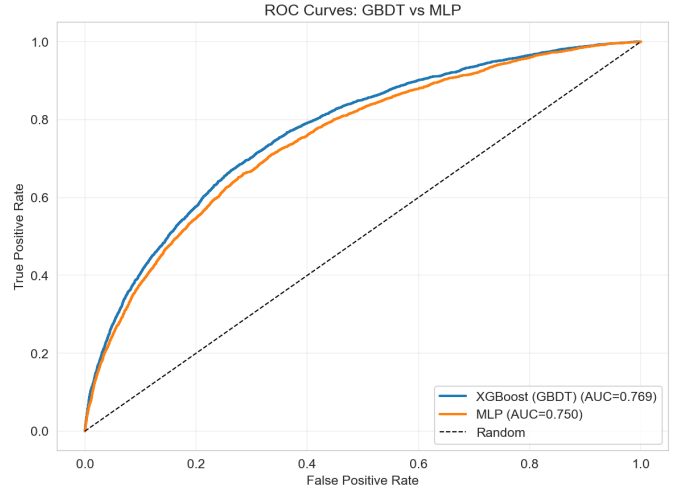


Figure 11: ROC curves. GBDT achieves $\text{AUC}=0.770$ vs. MLP’s 0.750. Both significantly outperform random, but the gap is smaller than in the PR space because ROC is less sensitive to class imbalance.

4.1 Threshold Analysis

The 0.5 decision threshold is a highly misleading operating point for an 8% minority class—neither model functions useably there. To identify a more realistic operating point, the F1-optimal threshold was computed on the *validation set* for each model and then applied to the test set. GBDT’s F1-optimal threshold was 0.153 (val F1=0.320); MLP’s was 0.178 (val F1=0.294).

Figure 12 contrasts the confusion matrices at both thresholds. Lowering the threshold substantially increases recall (true positive detection) at the cost of precision—exactly the trade-off a lender must navigate. At the F1-optimal threshold, GBDT identifies dramatically more defaulters, confirming that the model *has* learned the minority-class signal; the default 0.5 threshold simply suppresses it.

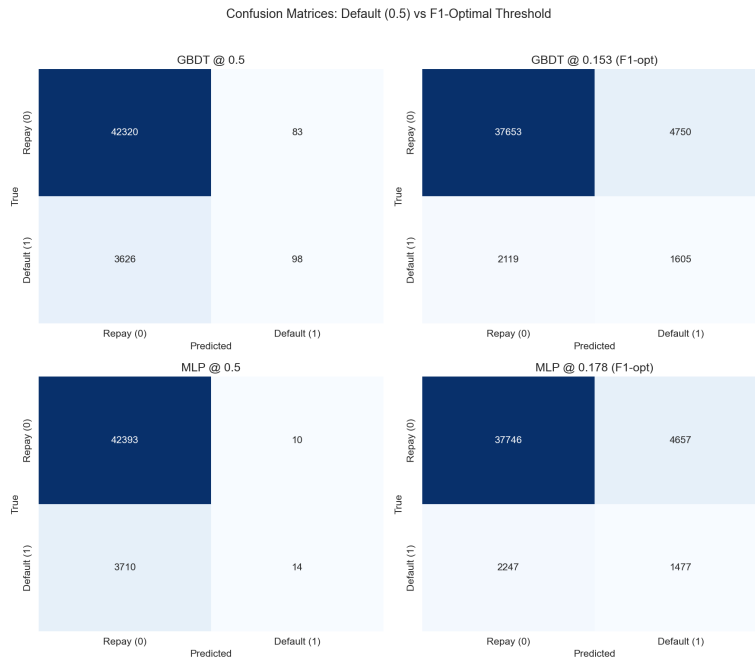


Figure 12: Confusion matrices at the default 0.5 threshold (left column) vs. the F1-optimal threshold (right column) for GBDT (top) and MLP (bottom). The F1-optimal threshold reveals substantially more true positive detections for both models.

5 Discussion

GBDT’s advantage on tabular data. The results confirm the well-documented finding that gradient boosted trees outperform neural networks on tabular data with moderate dimensionality and mixed feature types. GBDT’s axis-aligned splits are naturally suited to exploiting the strong, interpretable features in this dataset (external credit scores, financial ratios). The MLP’s dense weight matrices struggle to extract equivalent signal from the sparse one-hot encoded categoricals and the imputed features.

Bias-variance dynamics. Early stopping served as a dynamic regularizer for both models but manifested differently. The GBDT loss curve showed a gradual, controlled divergence between training and validation loss—the ensemble accumulated variance slowly as each new tree targeted smaller residuals. In contrast, the MLP converged within ~ 20 epochs and showed minimal train-validation gap, indicating *underfitting* rather than overfitting. The MLP’s capacity, while theoretically sufficient, was constrained by the challenging optimization landscape of the imbalanced, high-dimensional feature space.

Hyperparameter sensitivity. The MLP proved dramatically more sensitive to hyperparameter choices. Increasing the learning rate from 0.001 to 0.1 caused complete collapse ($\text{AUC-PR} \rightarrow 0.081$), while GBDT maintained AUC-PR above 0.244 across all learning rates. The tuned GBDT selected `learning_rate=0.1`, `max_depth=3` (shallower than baseline) with stronger regularization (`reg_alpha=1.0`, `reg_lambda=2.0`) and more aggressive subsampling (`subsample=0.6`), demonstrating that simpler trees combined with more boosting rounds generalize better—a bias-reduction strategy. Architecture and regularization changes in the MLP produced only marginal AUC-PR shifts (0.222–0.227).

Interpretability. GBDT provides direct feature importance by decomposing predictions into a sum of independent, axis-aligned decisions on individual features—each with a quantifiable contribution (gain) to the objective. It revealed that `EXT_SOURCE_MEAN` and the missing indicators are the dominant predictive signals. The MLP, in contrast, constructs distributed, entangled representations through nested compositions of non-linear functions. No feature exists in isolation; its contribution is co-determined by the values of all other features through dense weight matrices and conditional activations. This makes native feature attribution mathematically intractable without additional post-hoc methods (e.g., SHAP).

Practical considerations. GBDT trained $\sim 2.4\times$ faster (8.2s vs. 19.7s) while achieving better performance. GBDT also natively handles missing values and does not require feature scaling, reducing preprocessing overhead. For this credit risk domain—where interpretability and reliability are paramount—GBDT is the clear choice. MLP would be preferred for high-dimensional, unstructured data (images, text) where dense representations excel.

Limitations. The analysis was limited to the main application table; incorporating supplementary tables (bureau, installments, etc.) would likely improve both models. The `sklearn MLPClassifier` is a simplified implementation; modern frameworks (PyTorch) with batch normalization, dropout, and learning rate schedulers would likely improve MLP performance. No training-time imbalance correction was applied to either model to keep the comparison controlled, but real-world deployment would benefit from techniques like SMOTE or cost-sensitive learning.

6 AI Tool Usage Disclosure

- **AI Tools Used:** Gemini Pro and Antigravity.
- **AI Contribution:** Code generation for the preprocessing pipeline and notebook structure, dataset exploration, and LaTeX formatting support.
- **Personal Contribution:** All strategic decisions—feature selection, encoding strategy, imbalance handling philosophy, missing indicator design, experimental methodology—and the final analysis and narrative are the author’s own work.